CITYPILOT IA

Taller integrador: «¿Qué hacemos hoy?»

Duración: 150 minutos · **Equipos:** mínimo 2 (recomendado 2) · **Entregable:** repositorio GitHub + 2 Pull Requests por estudiante **y una memoria IA individual por persona** · **Modalidad: SEMÁFORO AMARILLO** (IA permitida como ayuda, uso transparente y justificable).

1. La historia

Una pequeña empresa de turismo quiere lanzar un prototipo: su visitante escribe una ciudad y lo que le apetece hacer —por ejemplo, una ruta cultural de cuatro horas por Valencia con niños—. El sistema consulta el tiempo atmosférico actual y localiza lugares reales; después pide a un modelo generativo que sugiera un itinerario razonado. Cada petición queda registrada y la empresa observa qué ciudades y actividades interesan más mediante un dashboard. Tenéis que entregar un prototipo **utilizable de principio a fin**.

El desafío no es construir un chatbot aislado: es conectar correctamente tres servicios propios (web, PostgreSQL, Jupyter), tres consultas de datos públicos y un modelo de lenguaje mediante una API Flask.

2. Qué debe ver el usuario al terminar

Pantalla 1 — Planificador. Formulario con ciudad (texto libre), tipo de turismo (cultural, gastronómico, naturaleza o familiar), duración (2, 4 o 6 horas) y preferencias en texto libre (opcional: «con niños», «evitar caminar mucho»). El botón genera una página con resumen del clima, varios lugares propuestos y una ruta narrativa elaborada por Cohere. **Los lugares proceden de OpenStreetMap, no de la**

imaginación del modelo. Mostrad un mensaje claro si estáis usando respuestas de prueba; no presentéis horarios de apertura ni eventos como verificados.

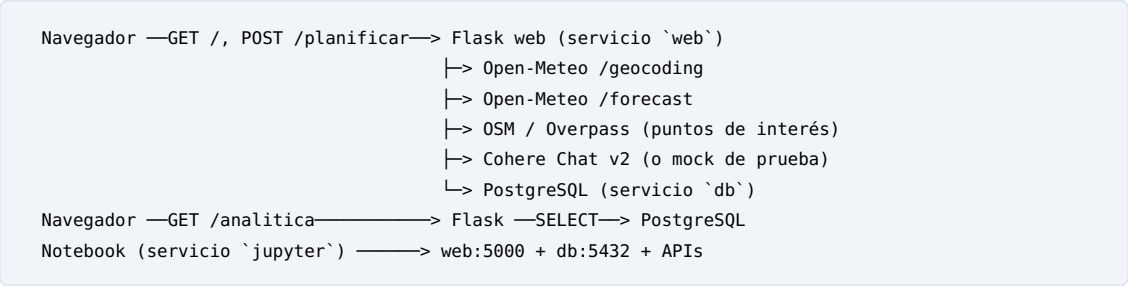
Pantalla 2 — Analítica. Panel con número total de peticiones, ciudades más consultadas, tipos de turismo, una tabla con las últimas 8 solicitudes y al menos **una gráfica generada desde PostgreSQL** (por ejemplo, barras por ciudad o tipo). Puede renderizarse como imagen PNG de Matplotlib: **no hace falta JavaScript**. Debe actualizarse al realizar nuevas consultas.

API propia. `POST /api/planificar` recibe las mismas preferencias en JSON que el formulario HTML. `GET /api/estadisticas` ofrece las métricas en JSON. Ambas interfaces utilizan **el mismo orquestador Python**, sin duplicar llamadas ni reglas.

PostgreSQL. Guardar en la tabla facilitada: entrada del usuario, fecha, ciudad, tipo, duración, preferencias, coordenadas, meteorología, lugares usados (JSON), prompt enviado, respuesta del modelo, estado, indicador de datos de prueba, modo de modelo y duración de la petición; registrar también fallos de forma comprensible. **No guardar claves API.**

Jupyter. Cuatro pruebas explícitas: servicio meteorológico, API Flask, lectura PostgreSQL y error de validación. No basta con arrancar Jupyter: deben verse pruebas ejecutadas en el notebook.

3. Arquitectura a implementar



Servicios de Docker Compose: **`web`, `db` y `jupyter`**. Ya existe un chasis con Dockerfiles, dependencias y SQL de inicio; no tenéis que perder media clase montando infraestructura desde cero, pero sí demostrar que comprendéis por qué `localhost` no sirve para comunicarse entre contenedores.

4. Lo que se os da vs. lo que hacéis

Parte	Ya suministrado	Debéis implementar
Docker	`compose.yaml`, Dockerfiles, `.env.example`	Levantar, verificar, solucionar incidencias y documentar
API pública	URLs, parámetros, contratos, ejemplos de JSON	Llamadas HTTP, validación, limpieza y gestión de errores
Modelo	Configuración Cohere + simulador para test	Prompt propio y llamada real Chat v2 con la clave docente
Base de datos	Tabla `solicitudes`, conexión y ping	INSERT de cada petición, consultas agregadas y últimos registros
Flask	6 rutas esqueleto, vista base, estilos CSS	Formulario, API JSON, orquestación, resultados y analítica
Jupyter	Notebook con enunciados de cuatro controles	Completar y ejecutar celdas con evidencias
GitHub	Instrucciones de flujo y reparto sugerido	2 PR por integrante, revisión cruzada y merges
Semáforo amarillo	Reglas y plantilla de memoria individual	Evidencia real de uso de IA y fuentes, decisiones, pruebas y aportación propia

En cada fichero hay `TODO(ALUMNADO)`. **No se proporciona la solución**, ni el prompt ni el HTML completo. El archivo **`docs/TAREAS_Y_CRITERIOS.md`** os detalla qué debe hacer CADA función, sus entradas/salidas, errores y cómo comprobarla, pero sin escribir el código. Se valora que toméis decisiones justificadas y podáis explicar el código, aunque hayáis usado herramientas de IA como asistencia. Las ayudas sustantivas se registran individualmente en la memoria obligatoria.

5. Plan sugerido para 150 minutos

Minutos	Hito	Señal de que podéis pasar al siguiente
0-15	Equipo, repo, `.env`, Compose y reparto	`/api/health` responde; Jupyter abierto; ramas acordadas
15-40	**PR 1 de cada persona:** servicios externos	Ciudad → coordenadas, clima y lista normalizada de lugares; 2 PR abiertos y revisados
40-75	**PR 2 de cada persona:** HTML, Cohere, DB y panel (repartir bien)	Flujo funcional aunque sea en mock; 4 PR en curso o fusionados
75-110	Integración y mejoras: validaciones, resultado, API JSON, dashboard	Una petición aparece en DB y en pantalla; 1 gráfica real
110-135	Jupyter, pruebas y correcciones de PR	4 validaciones del notebook; >=5 solicitudes de prueba
135-150	Evidencias, cierre de memoria y entrega GitHub	`main` funcional, 2 PR/autor fusionados, README final y 1 memoria IA por persona

El tiempo es orientativo, no una excusa para ampliar el alcance. Primero completad el circuito mínimo; si os sobra tiempo, elegid bonus. Las tareas pueden solaparse. No dediquéis 40 minutos a pulir CSS antes de tener datos reales.

6. Desarrollo paso a paso

Fase A — Datos externos. Implementad las funciones `geocodificar`, `obtener_meteorologia` y `obtener_lugares` en `api_clients.py`. Las tres reciben y devuelven diccionarios/listas normalizados. Para la ciudad filtrad a España si os ayuda a reducir ambigüedad; para Overpass trabajad con un radio acotado de 2-3 km. El fichero `docs/APIS_Y_CONTRATOS.md` indica URLs, parámetros y estructura de respuesta. Diferenciad ausencia de resultados de fallo HTTP; y poned timeout.

Fase B — Prompt y Cohere. En `llm.py` desarrollad vuestra plantilla de prompt (no se entrega) **y su llamada HTTP**. No se facilita el texto de un prompt completo ni el código final de consumo de Cohere. Enviad: ciudad, preferencias, duración, clima y lista de destinos confirmados, con coordenadas. Pedid una ruta de 3-5 paradas, razonada y adaptada a posibles precipitaciones. El prompt debe **prohibir** inventar lugares o decir que un local está abierto cuando no se ha comprobado. **Probádlo**: contrastad al menos una respuesta con los datos enviados y anotad en la memoria la versión inicial, sus resultados y una mejora que hayáis evaluado (si no hubo cambio, justificadlo). La clave se configura exclusivamente en `.env`. Podéis probar el flujo general con `LLM_MODE=mock`, pero el requisito completo exige una llamada auténtica al modelo facilitado en clase.

Fase C — Integración. `procesar_solicitud` de `orchestrator.py` es el centro del sistema. Debe validar, ejecutar las etapas en secuencia, medir el tiempo, gestionar errores, almacenar resultado y devolver un único diccionario consistente. Las dos rutas Flask (HTML y JSON) llaman a esa función. **No hagáis que el HTML consulte APIs directamente.**

Fase D — Front y API propia. Escribid el `<form>` del archivo `index.html` con nombres de campo coherentes con el contrato. Implementad `POST /planificar` para recoger `request.form`, y `POST /api/planificar` para recoger `request.get_json()`; ambos validan y usan el mismo orquestador. Rellenad `resultado.html` con el clima, lugares y

texto devuelto por el modelo. Mostrad si el resultado es real o simulado; podéis añadir enlaces de cada lugar al mapa de OpenStreetMap usando sus coordenadas, sin incorporar JavaScript.

Fase E — Persistencia y analítica. La tabla SQL ya está creada. Implementad `guardar_solicitud`, `obtener_estadisticas` y `obtener_ultimas_solicitudes` usando parámetros SQL y SQLAlchemy; se pueden usar sentencias SQL sencillas. En `analitica.html` sustituid los guiones por métricas de la DB, incorporad una tabla con 8 registros y completad `/analitica/grafico.png` con Matplotlib. Al enviar una nueva solicitud, refrescar la página debe cambiar los resultados.

Fase F — Notebook y validación. Abrir `01_validacion.ipynb` dentro del contenedor `jupyter` y resolver sus cuatro ejercicios. Las llamadas HTTP a Flask desde el notebook se hacen a `http://web:5000`; las consultas SQL usan `db:5432`. Añadid también al menos **dos tests automatizados propios** al proyecto (`pytest`) para errores, contratos o normalización (se suministran 2 pruebas de humo, que no cuentan como las vuestras).

7. Riesgos reales que debéis contemplar

Overpass puede estar saturado. No hagáis llamadas cada vez que se recarga el panel. Un timeout y mensaje útil son preferibles a bloquear la app. Para probar el pipeline podéis recurrir a `fixtures/puntos_demo.json`, claramente etiquetado como sintético; no hace falta implementar un sistema de reintentos complejo. Los datos de respaldo NO sustituyen la demostración de una petición externa cuando el servicio está disponible.

Cohere necesita clave. Aunque tenga evaluación gratuita, requiere cuenta/credencial. El docente proporciona una clave para la sesión. La opción mock evita que el alumnado tenga que abrir cuentas y permite hacer la práctica si se agota el cupo; no equivale a consumo real de Cohere.

«Qué hacer hoy» no es «qué eventos hay hoy». Estamos construyendo una ruta adaptada al clima actual y a lugares cartografiados, no una agenda de eventos o un verificador de horarios. La interfaz debe avisarlo. Tampoco se calculan tiempos reales de desplazamiento; cualquier orden de visita es una sugerencia, no navegación GPS verificada.

Seguridad y atribución. Contraseñas y claves fuera de Git, consultas SQL parametrizadas, no mostrar stack traces al usuario, evitar mostrar HTML sin escapar y citar Open-Meteo y © contribuyentes OpenStreetMap. Todo el proyecto es demostrativo, no está preparado para producción.

8. Semáforo amarillo: uso de IA y memoria OBLIGATORIA

En esta actividad **podéis utilizar IA para aprender, depurar, explicar problemas, explorar soluciones y revisar vuestro código**, pero el proyecto y las decisiones deben ser defendibles. No se os pide esconder que habéis recibido ayuda; **se os pide documentarla y no delegar ciegamente toda la actividad**. Tampoco se considera honesto atribuirse código copiado de GitHub, documentación o un compañero sin reconocerlo.

La memoria no es un anexo decorativo: es un entregable individual y se evalúa. Cada estudiante creará una copia de `entrega/MEMORIA_IA_PLANTILLA.md` y la guardará como `entrega/MEMORIA_IA_<NOMBRE>.md`. En ella deben figurar su aportación, herramientas y modelos consultados, prompts o instrucciones relevantes, respuestas/sugerencias obtenidas (también intentos fallidos), qué se incorporó o descartó, cómo se comprobó y vínculos con commits, PR y tests. Es especialmente importante documentar la **evolución del prompt de la aplicación** y los fragmentos de código propuestos por IA o extraídos de otras fuentes. Si alguien no utilizó IA, presentará igualmente su memoria indicando esa circunstancia y sus fuentes y evidencias de trabajo propio.

Una anotación como «usé ChatGPT para ayudarme» es insuficiente. Lo útil es: «tenía el error X; pregunté Y; el modelo recomendó Z; descarté una parte porque...; cambié esta función; lo comprobé con este test; aparece en PR #...». No es necesario capturar cada pregunta trivial, pero sí las interacciones sustantivas. **No incluyáis nunca credenciales** en los prompts, capturas o documentos de entrega.

Posible reducción adicional: una memoria manifiestamente falsa, que oculte aportaciones sustantivas de IA/terceros o invente pruebas o contribuciones, puede conllevar **hasta un 30 % de reducción de la nota final de ESTA actividad**, adicional a los puntos perdidos en el criterio de memoria. No se aplica por utilizar IA ni por una sospecha basada solo en estilo o detectores automáticos: se contrastarán me-

moria, PR, commits, resultados y explicaciones personales y se solicitará aclaración antes de tomar una decisión. La memoria omitida obtiene 0 en su criterio, pero no equivale automáticamente a falsearla.

Las instrucciones completas, los ejemplos y la plantilla están en `docs/SEMAFORO_AMARILLO_Y_MEMORIA.md` y `entrega/MEMORIA_IA_PLANTILLA.md`. **Id anotando las evidencias durante el desarrollo**, no las inventéis al final.

9. Entrega y rúbrica (10 puntos)

Entregad UN enlace al repositorio GitHub, con `main` funcional, README propio (arranque, qué funciona, qué falta), dos capturas (resultado y panel), notebook ejecutado, **dos PR revisados por integrante** y **una memoria IA `.md` por integrante** dentro de `entrega/`. Los PR deben identificar autores y revisores. La entrega debe permitir comprobar consultas SQL, pruebas y código sin acceder a secretos.

Criterio	Puntos	Qué se observa
APIs externas	1,4	Geo + clima + Overpass; datos normalizados, control de errores
Cohere y prompt	1,1	Llamada real si hay clave, prompt propio, datos verificables y adaptación
Flask y orquestador	1,5	Formulario, API JSON, validación, resultado y lógica compartida
Persistencia y analítica	1,6	INSERT completo, dashboard de PostgreSQL, tabla y gráfica
Docker, Jupyter y tests	1,3	3 contenedores, notebook con evidencias, 2 tests nuevos
Git, PR y colaboración	1,1	2 PR por persona, revisión cruzada, cambios reales y atribuibles
Memoria IA individual, veraz y contrastable	**2,0**	Registro 0,8; criterio y decisiones 0,6; evidencias 0,4; reflexión personal 0,2
TOTAL	**10,0**	

Bonus hasta +1 (máximo final antes de posibles penalizaciones: 10): enlaces individuales a OSM (+0,25), comparación lluvia/sol documentada (+0,25), cache de Overpass (+0,25), gráfico extra por ciudad/tipo (+0,25). Los bonus no sustituyen requisitos obligatorios.

Cómo opera el 30 %: tras obtener la nota por rúbrica y bonus (máximo 10), se puede aplicar una reducción proporcional motivada de **0-30 %** si se comprueba falsedad relevante. Ejemplo: 8,0 y reducción del 20 % → 6,4. **No es una penalización del 30 % sobre toda la asignatura ni una penalización automática por usar IA.**

10. Criterios de éxito observables

Al terminar, la pareja puede enseñarme: **(1)** tres servicios levantados; **(2)** una ciudad y un tipo insertados desde la web; **(3)** respuestas actuales de geocoding y meteorología; **(4)** puntos OSM reales y la adaptación de la ruta por IA; **(5)** una fila completa en Postgres; **(6)** un dashboard que cambia tras otra consulta; **(7)** notebook con pruebas ejecutadas; **(8)** cuatro PRs con revisión cruzada; **(9)** memoria individual de IA honesta y verificable por cada estudiante. No se premia escribir más líneas, sino cerrar el ciclo y comprenderlo.

Documentación de apoyo: `docs/APIS_Y_CONTRATOS.md`, `docs/TAREAS_Y_CRITERIOS.md`, `docs/GIT_Y_PRS.md`, `docs/COMPROBACIONES.md`, `docs/SEMAFORO_AMARILLO_Y_MEMORIA.md` y `entrega/MEMORIA_IA_PLANTILLA.md`.